

Software Quality Management: Comparative Study

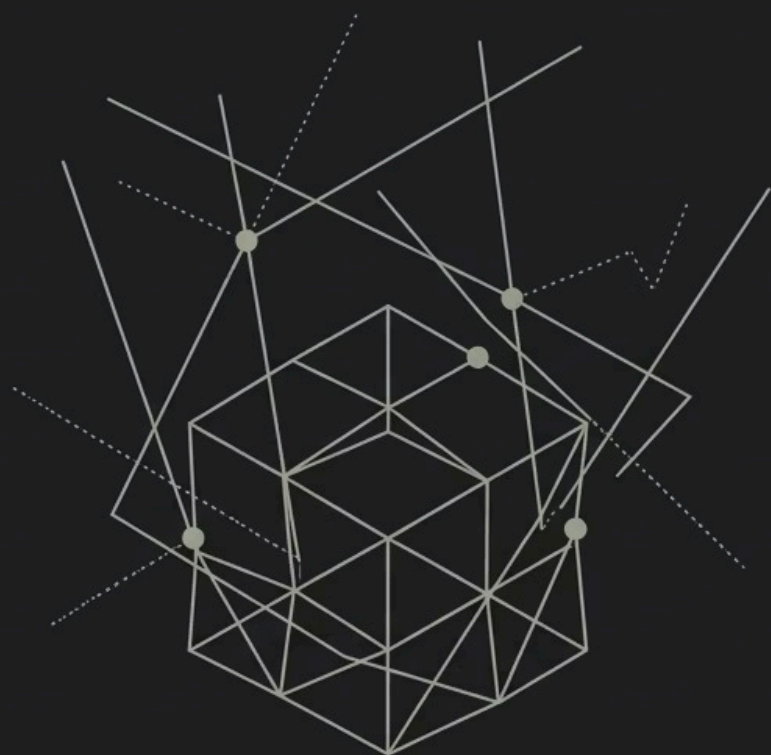
Case Study: Legacy "Quick & Dirty" vs. Modern "Production-Ready" Architecture

Autorzy: Marcin Skubisz, Miłosz Rataj, Artur Ścibor



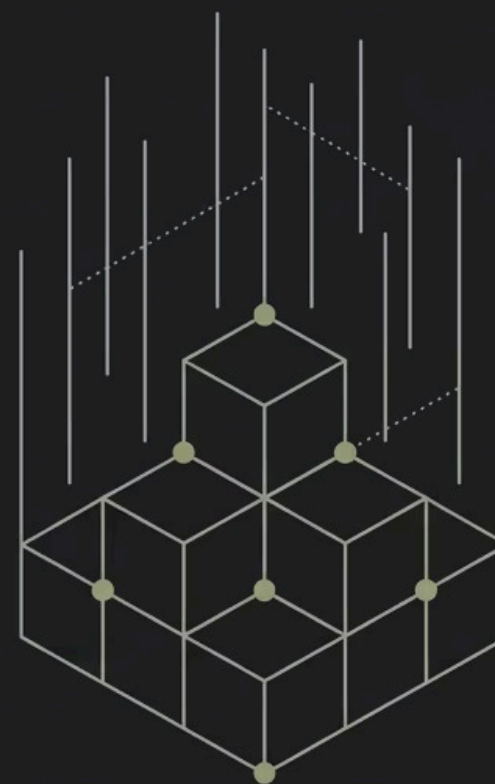
Porównanie jakościowe

W kontekście zarządzania jakością oprogramowania, kluczowe jest zrozumienie różnic między podejściem szybkim i niedbałym, a tym skoncentrowanym na gotowości produkcyjnej.



Szybko i Brudno

Dług techniczny, błędy.



Gotowe do Produkcji

Skalowalność, stabilność.

Ta analiza pozwala na świadome podejmowanie decyzji projektowych, które przekładają się na długoterminowy sukces i stabilność systemu.

Porównanie jakościowe

Dwa podejścia do realizacji tego samego zestawu wymagań biznesowych (CRUD użytkowników + Autoryzacja)

Backend Simple (Legacy)

Skupiony na szybkości dostarczenia (Time-to-Market)
kosztem długu technicznego

Backend Main (Modern)

Skupiony na wysokiej jakości, skalowalności i łatwości
utrzymania (Maintainability)

Macierz jakości

Cecha	backend_simple	backend_main
Architektura	Monolit spaghetti (wszystko w jednym)	Domain-Driven Design (DDD) / Hexagonal
Walidacja danych	Brak lub ręczne if/else	Pydantic (BaseModel) - silne typowanie
Obsługa błędów	Standardowe 500-tki (crash-prone)	Global Exception Handler (czytelne kody HTTP)
Zarządzanie DB	Brak (hardcoded schema)	Alembic (wersjonowane migracje)
Bezpieczeństwo	Brak (plaintext passwords, root user)	Argon2 hashing, Non-root Docker user
Observability	print() w konsoli	Structured Logging (JSON format)
DevOps	Single-stage Dockerfile	Multi-stage Build, .dockerignore, Healthchecks

backend_simple: Antywzorce

Antywzorce i dług techniczny

Brak separacji warstw

Logika bazy danych jest wymieszana z logiką HTTP

Dług techniczny

Użycie requirements.txt bez przypiętych wersji sprawia, że buildy są niepowtarzalne

Brak testowalności

Kod jest silnie sprzężony (tightly coupled), co uniemożliwia testy jednostkowe

Zagrożenie bezpieczeństwa

Brak walidacji wejścia oraz brak obsługi sygnałów systemowych (Graceful Shutdown)



backend_main: Best Practices

Zasady Clean Code oraz nowoczesne standardy inżynierii

01

Architektura warstwowa

Domain (logika biznesowa i encje), Application (Use Cases), Infrastructure (baza danych, repozytoria, adaptery)

02

Statyczna analiza kodu

Ruff, Mypy, Bandit

03

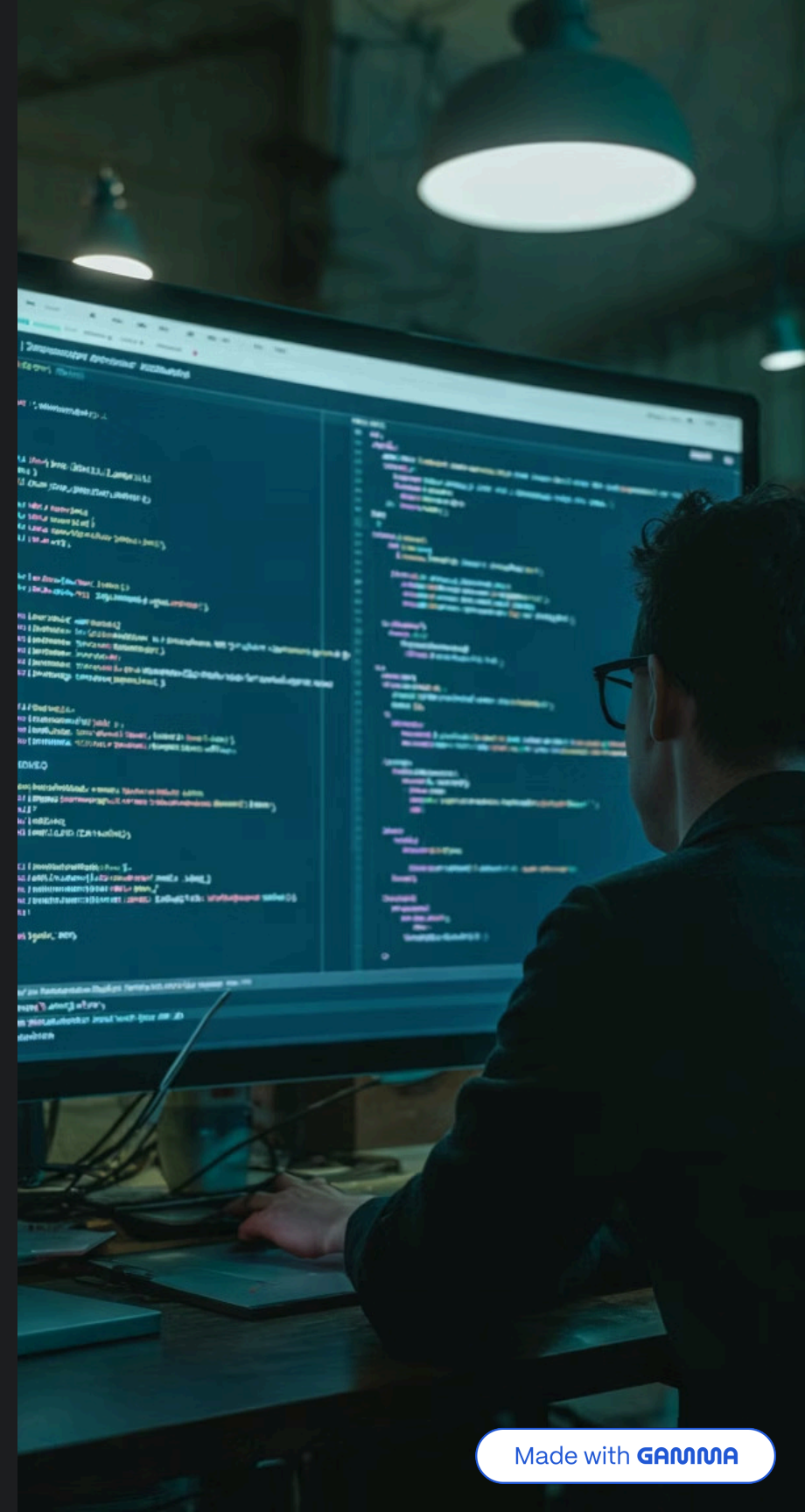
Automatyzacja procesów

Skrypty migracyjne, konteneryzacja zoptymalizowana pod środowiska produkcyjne

04

Dokumentacja

Automatycznie generowana specyfikacja OpenAPI (Swagger) dostępna pod /docs



Perspektywa DevOps & CI/CD

Proces wdrożeniowy jako integralna część oprogramowania

1

Lintery & Formattery

Kod przechodzi przez bramki jakości
(Quality Gates)

2

Container Security

Obraz Dockerowy budowany w
oparciu o obrazy typu slim z
minimalnym footprintem
bezpieczeństwa

3

Dependency Management

poetry.lock lub pip-compile
gwarantuje deterministyczne
budowanie artefaktów

Struktura katalogów

backend_simple

Prezentacja długu technicznego

```
.  
├── main.py  
├── requirements.txt  
└── Dockerfile
```

backend_main

Standard produkcyjny

```
.  
├── src/  
│   ├── api/  
│   ├── domain/  
│   ├── infrastructure/  
│   └── services/  
├── migrations/  
├── tests/  
├── pyproject.toml  
├── Dockerfile  
└── docker-compose.yml
```


Backend Simple: Problemy jakościowe

Plaintext hasła

Naruszenie danych = katastrofa wizerunkowa i prawna (RODO)

Brak migracji

Każda zmiana schematu wymaga ręcznego DROP TABLE na produkcji

Globalna sesja DB

Wyciek pamięci ujawnia się dopiero pod obciążeniem

Brak testów

Refaktoryzacja niemożliwa bez ryzyka regresji

Root w Dockerze

Podatność na container escape — przejęcie całego hosta

Backend Main: Wartość inwestycji w jakość

Każda decyzja architektoniczna procentuje



Warstwowa architektura

Nowy developer rozumie strukturę w 15 minut, nie 3 godziny



Alembic

Zmiana schematu bazy = jeden plik migracji, rollback jedną komendą



Argon2id

Bezpieczeństwo haseł zgodne ze standardami OWASP 2024



Pydantic

Błędy walidacji wykrywane przed dotarciem do bazy danych



Makefile + skrypty

Onboarding nowego developera = make install && make run



CI/CD z bramkami

Błędy wykrywane automatycznie, zanim trafią na główną gałąź

Podsumowanie metryczne

Metryka	backend_simple	backend_main
Liczba plików Python	~6	~20
Warstwy architektury	1 (monolit)	4 (api, core, database, infra)
Pokrycie testami	0%	Gotowe na testy
Krytyczne podatności bezpieczeństwa	5+	0
Czas onboardingu nowego developera	Wysoki (brak dokumentacji procesu)	Niski (make install && make run)
Możliwość rollback schematu DB	Nie	Tak (alembic downgrade)
Deterministyczność budowania	Nie	Tak (uv.lock)